

PD Module 2 — Lab Manual: Data Preparation

After completing the lab, you will be able to,

- 1 Set up work area
- 2 Invoke the tool
- 3 Read all the required inputs (known as Data setup)
 - a Create Library
 - b Read reference library files
 - c Open .lib and .lef files
 - d Read the Technology file
 - e Open technology file
 - f Read Netlist file
 - g Open Netlist file
 - h Read SDC file
 - i Open SDC file
 - j Read TLU+ files
- 4 Link the design
- 5 Save the block

Task

Copy the lab database to your home directory.

```
linux > cp -rf /home/chiptop2.0/pd ./
```

Go through the directory structure & files.

RTL -> Directory for RTL files.

inputs -> Directory for inputs file.

logs -> Directory for log files

reports -> Directory to save all reports

scripts -> Directory for script files

outputs -> Directory to save output files

work -> Directory to run the tool

In the scripts directory open, "data_preparation.tcl" using "vi" or "gvim" editor command from the scripts directory and go through the file contents.

```
linux > gvim data_preparation.tcl
```

From a different linux shell, Invoke the lcc2 Compiler tool using "icc2_shell" from the work directory by following steps,

```
linux > cd chiptop/pd
linux> pd > cd work
linux> work > tcsh
linux> work > icc2_shell -output_log_file ../logs/data_prep.log
```

Creating Library:

Reference libraries for the IC Compiler II tool use the new data model (NDM) format. These reference libraries are unified libraries that contain the logical and physical information for a specific technology and one or more of its library cells. Each library cell is represented as a single object with multiple views that contain various types of information about the cell.

A library cell can have the following views:

Timing view

The timing view contains the timing, power, and functional information for a cell. It can contain timing and power information for multiple characterization points (voltage and temperature) for a given process setting. The data for each characterization point is stored in a pane. The IC Compiler II tool uses the timing view to perform static timing analysis, power analysis, and optimization.

Activity 1:

```
Let's open one more terminal,
icc2_shell > sh xterm &
The new terminal will be a linux shell.
In the new terminal Open the .lib file,
Linux>gvim /app/home/tools/14nm/stdcell_hvt/db_nldm/
saed14hvt_ff0p88v125c.lib
```

1. Grep for “cell”, look for different standard cells.
2. Check for the “AREA” of each standard cell.
3. Check for Leakage power and Function.

Layout view

The layout view contains the physical shape information for a cell, not including connectivity and pins. The library manager uses the layout view to create the design view from GDSII or Open Artwork System Interchange Standard (OASIS) source libraries, and the IC Compiler II tool uses it for mask generation.

Activity 2:

Let's look into the contents of layout file (.lef)

```
Linux > gvim /app/home/tools/14nm/stdcell_hvt/lef/
saed14nm_hvt_1p9m.lef
```

Design view

The design view contains the physical shape information for a cell, including connectivity and pins. The library manager uses the design view to create the frame view.

Frame view

The frame view contains the information needed to perform placement and routing, such as the block boundary, pins, via regions, and routing blockages. The IC Compiler II tool uses the frame view to perform placement and routing.

It is necessary to create a library in which the work will be performed.

Now copy the commands from the "data_preparation.tcl" file and paste each command one by one in the "icc2_shell" command prompt to start the Data preparation lab as shown below. Set the current project path, link library path, target library path for the design.

Now specify directories that the tool searches for files, by using the below command.

```
icc2_shell>source ../scripts/common_setup.tcl
```

Above command sets the link library to be used by the ICC2 to resolve references which are instantiated in the RTL. These could be either PLL, Memory, standard cell, or any other macro for which there is no module definition present in the RTL. For example,

```
BUF uBUF (.FIN(clk), .FOUT(clkout));
```

The "BUF" module shall be resolved from the link library. That is, the DC compiler investigates the link library in order to get the characteristic information of the BUF like timings etc.

Specify the list libraries used in the design, by using the "link_library" command.

```
icc2_shell>set link_library $LINK_LIBRARY_FILES_CLG
```

Specify the list of technology libraries, by using the "target_library" command.

```
icc2_shell>set target_library $TARGET_LIBRARY_FILES_CLG
icc2_shell>create_lib          -ref_libs
$NDM_REFERENCE_LIB_DIRS_CLG   -technology $TECH_FILE
../lib/chiptop
```

Technology file

The technology file contains information about the routing layers and routing rules for a specific technology.

Activity 3:

Open the technology file and check the contents of it.

```
Linux> gvim /app/home/tools/14nm/tech/new_tf/
saed14nm_1p9m_mw.tf
```

Check:

1. Units used for voltage, current, power and resistance.
2. Details related to Layer M1, color, min width, max width, pitch.

Reading Netlist file:

```
icc2_shell> read_verilog ../inputs/ChipTop_phy.v
```

Activity 4:

Open the netlist file ChipTop_phy.v using "gvim" command.

```
Linux> ChipTop_0901/pd/inputs > gvim
../inputs/ChipTop_phy.v
```

Go to the end of the file, grep for the module, in the verilog netlist file the last module defined is the top design.

1. Check the ports declaration for Inputs and outputs.
2. Look for Macro and Standard cell instantiations.

Setting the current design:

```
icc2_shell > current_design ChipTop
icc2_shell > link
```

Saving the Library:

When you create or change a library, the changes are stored in memory only. To save a library to disk, use this command.

```
icc2_shell > save_lib -as post_import_design
```

Activity 5:

```
icc2_shell > man link
```

Study the manual page and understand what the command does.

Defining Routing directions to Metal layers:

```
icc2_shell > define_user_attribute -type string -name  
routing_direction -classes routing_rule  
icc2_shell > set_attribute -objects [get_layers {M1 M3 M5  
M7 M9}] -name routing_direction -value vertical  
icc2_shell > set_attribute -objects [get_layers {M2 M4 M6  
M8 MRDL}] -name routing_direction -value horizontal  
icc2_shell > set_wire_track_pattern -site_def unit -layer  
M1 -mode uniform -mask_constraint {mask_two mask_one} -  
coord 0.037 -space 0.074 -direction vertical
```

Adding TLU+ files:

TLUPlus files

The TLUPlus files contain the RC coefficients for a specific technology. The TLUPlus models enable accurate RC extraction results by including the effects of width, space, density, and temperature on the resistance and capacitance.

```
icc2_shell > read_parasitic_tech -tlup  
/app/home/tools/14nm/tech/star_rc/max/saed14nm_1p9m_Cmax.tlup  
lus -layermap  
/app/home/tools/14nm/tech/star_rc/saed14nm_tf_itf_tluplus.map  
-name worst_para  
  
icc2_shell > read_parasitic_tech -tlup /app/home/tools/14nm/  
tech/star_rc/min/saed14nm_1p9m_Cmin.tluplus -layermap  
/app/home/tools/14nm/tech/star_rc/saed14nm_tf_itf_tluplus.map
```

Creating Multi Corner Multi Mode Analysis:

```
icc2_shell > source -e -v ../scripts/mcmm.tcl
```

Reading SDC file:

```
icc2_shell > set Constraints_file "../inputs/  
output.ChipTop_phy.sdc"
```

Activity 6 :

Open the sdc file `output.ChipTop_phy.sdc` using "gvim" command

Linux: `ChipTop_0901/pd/inputs > gvim output.ChipTop_phy.sdc`

Check for:

1. How many clocks are defined.
2. Defining false paths.
3. Defining Input and Output delays.

Removing All Existing Modes, Corners & Scenarios:

```
icc2_shell > remove_corners -all
icc2_shell > remove_modes -all
icc2_shell > remove_scenarios -all
icc2_shell > remove_propagated_clocks [get_ports]
icc2_shell > remove_propagated_clocks [get_pins
-hierarchical]
```

Creating Corners:

```
icc2_shell > create_corner slow
icc2_shell > create_corner fast
```

Reading Parasitic Files:

```
icc2_shell > read_parasitic_tech -tlup /app/home/tools/14nm/
tech/star_rc/min/saed14nm_1p9m_Cmin.tluplus      -layermap
/app/home/tools/14nm/tech/star_rc/saed14nm_tf_itf_tluplus.map
-name tlup_max

icc2_shell > read_parasitic_tech -tlup /app/home/tools/14nm/
tech/star_rc/max/saed14nm_1p9m_Cmax.tluplus      -layermap
/app/home/tools/14nm/tech/star_rc/saed14nm_tf_itf_tluplus.map
-name tlup_min
```

Setting Parasitic Parameters and temperature to corners

```
icc2_shell > set_parasitics_parameters \
    -early_spec tlup_max \
    -late_spec tlup_max \
    -early_temperature 125 \
    -late_temperature 125 \
    -corners {fast}

icc2_shell > set_parasitics_parameters \
    -early_spec tlup_min \
    -late_spec tlup_min \
    -early_temperature -40 \
    -late_temperature -40 \
    -corners {slow}
```

Creating Modes:

```
icc2_shell > create_mode func
icc2_shell > current_mode func
```

Creating Scenarios:

```
icc2_shell > create_scenario -mode func -corner fast -name
func_fast
icc2_shell > create_scenario -mode func -corner slow -name
func_slow
```

Reading Constraints w.r.t Scenarios

```
icc2_shell > current_scenario func_slow
icc2_shell > read_sdc $Constraints_file
icc2_shell > current_scenario func_fast
icc2_shell > read_sdc $Constraints_file
```

To enable or disable timing analysis, power analysis, and DRC checking for scenarios.

```
icc2_shell > set_scenario_status func_slow -none -setup
true -hold false -leakage_power true -dynamic_power true -
max_transition true -max_capacitance true -min_capacitance
false -active true

icc2_shell > set_scenario_status func_fast -none -setup
false -hold true -leakage_power true -dynamic_power false
-max_transition true -max_capacitance false -
min_capacitance true -active true
```

To remove duplicate modes, corners, scenarios, and to improve runtime and capacity without loss of constraints:

```
icc2_shell > remove_duplicate_timing_contexts
```

Creating Path Groups:

```
icc2_shell > group_path -name MACRO2REG -from [get_cells -
physical_context -filter design_type==macro] -to
[all_registers]
icc2_shell > group_path -name REG2MACRO -from
[all_registers] -to [get_cells -physical_context -filter
design_type==macro]

icc2_shell > group_path -name INPUTS -from [all_inputs]
icc2_shell > group_path -name OUTPUTS -to [all_outputs]
icc2_shell > group_path -name REG2REG -from
[all_registers] -to [all_registers ]
```

Save the Block:

```
icc2_shell > save_block -as import_done  
icc2_shell > save_lib
```

Closing the Block and Library:

```
icc2_shell > close_blocks -force  
icc2_shell > close_lib  
Icc2_shell > exit
```